FinWave

FinTech Financial Inclusion Platform for Migrant Workers

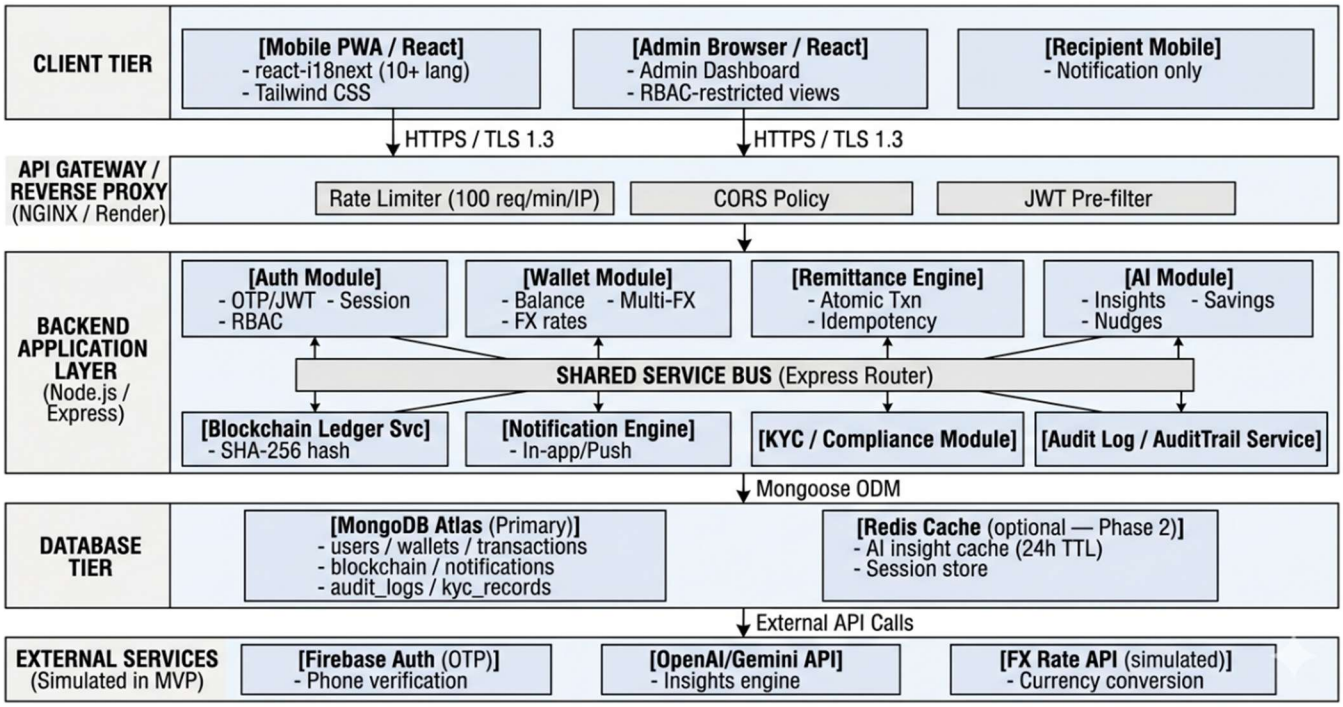
COMPLETE UML DESIGN SUITE

Enterprise-Grade Software Architecture Documentation

Project	FinWave — Financial Inclusion Platform
Team	Team 20
Document Type	UML Design Suite — 14 Diagrams
Standard	IEEE 830 / UML 2.5
Version	v1.0.0 — May 2026
Status	Hackathon Submission — Final

01 High-Level System Architecture Diagram

Modular Monolith · Cloud-Native · RESTful API Design

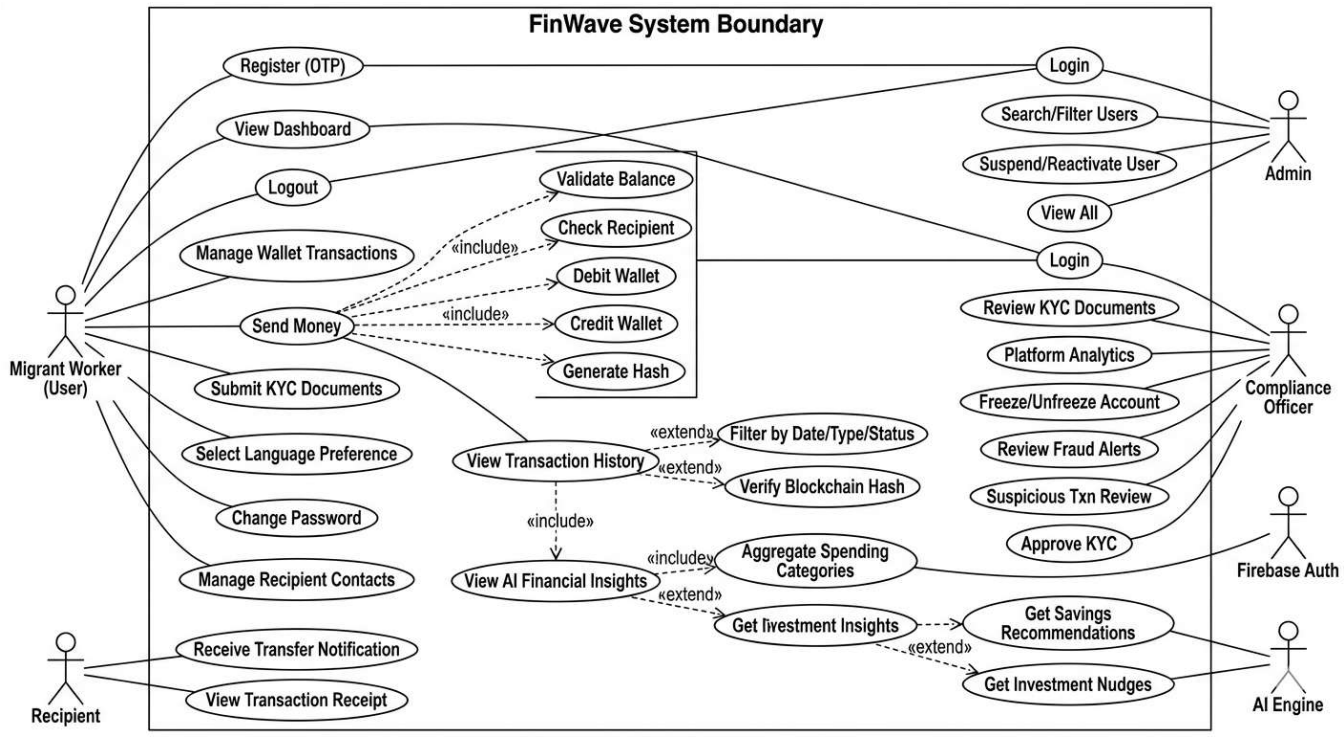


Key Design Decisions

Decision	Justification
Modular Monolith	All modules share one Node.js process — eliminates inter-service network latency. Ideal for 2-day hackathon timeline while remaining split-ready for microservices later.
API Gateway Pattern	Centralises JWT validation, rate limiting, and CORS before requests hit business logic — zero security duplication across services.
MongoDB Atlas	Schema-flexible NoSQL suits rapidly-evolving fintech data models. M0 free tier covers hackathon; scales to M10 with minimal code change.
Firebase Auth (OTP)	Eliminates custom OTP infrastructure. Production-grade SMS delivery via Firebase globally. Zero setup cost for hackathon.
AI via REST API	GPT-4o-mini / Gemini called over HTTPS — no ML infrastructure needed. 24-hour caching prevents rate limit hits during demo.
Stateless JWT Backend	Enables horizontal scaling (add more Render.com instances) without sticky sessions. RS256 signing prevents token forgery.

02 Use Case Diagram

Actors · Use Cases · include / extend Relationships

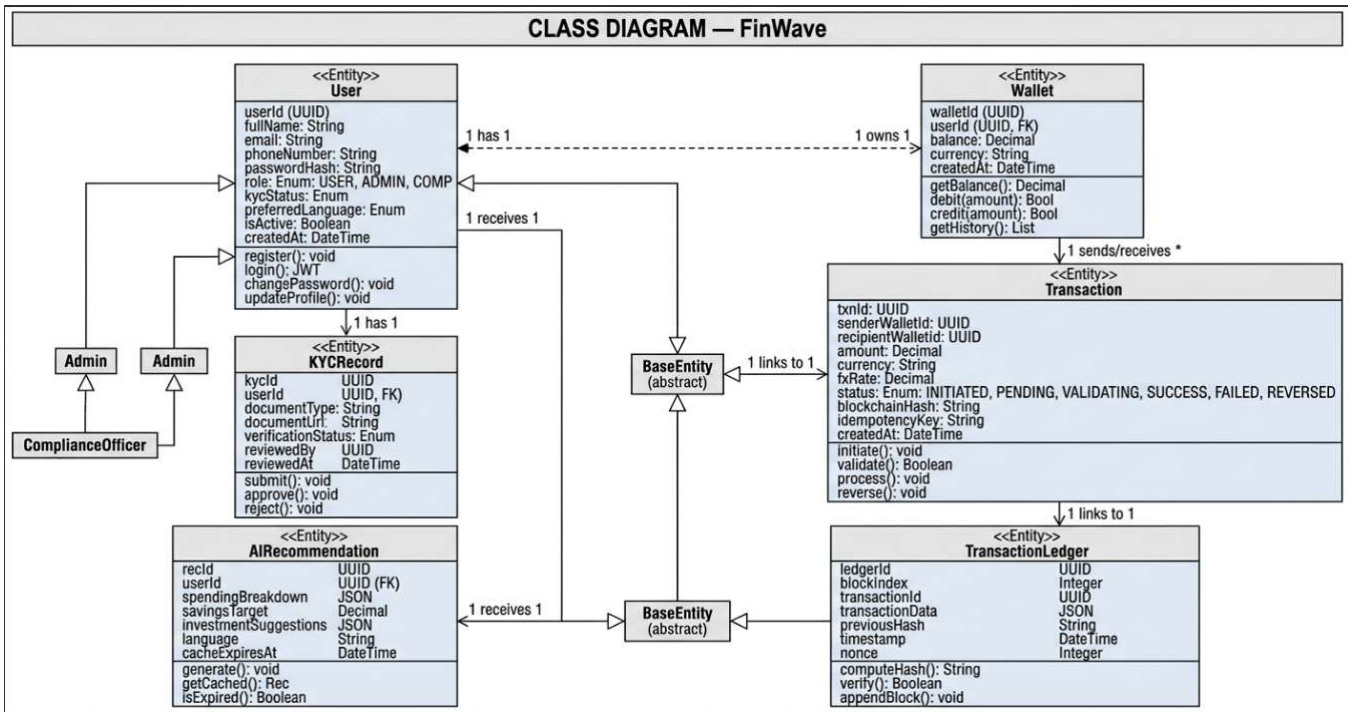


Actors and Their Roles

Actor	Description & Interaction Scope
Migrant Worker (User)	Primary actor. Registers, authenticates, manages wallet, sends/receives money, views AI insights, sets language preference.
Recipient (Beneficiary)	Passive actor — receives in-app/push notification on credit. May or may not be a registered FinWave user.
System Administrator	Manages users, reviews KYC, suspends/reactivates accounts, views platform-wide analytics.
Compliance Officer	Reviews fraud alerts, suspicious transactions, and KYC document submissions for regulatory compliance.
AI Engine (System)	Automated actor. Generates categorized insights, savings recommendations, investment nudges on schedule or demand.
Firebase Auth (External)	Handles OTP generation, verification, and Firebase token issuance for all authentication flows.

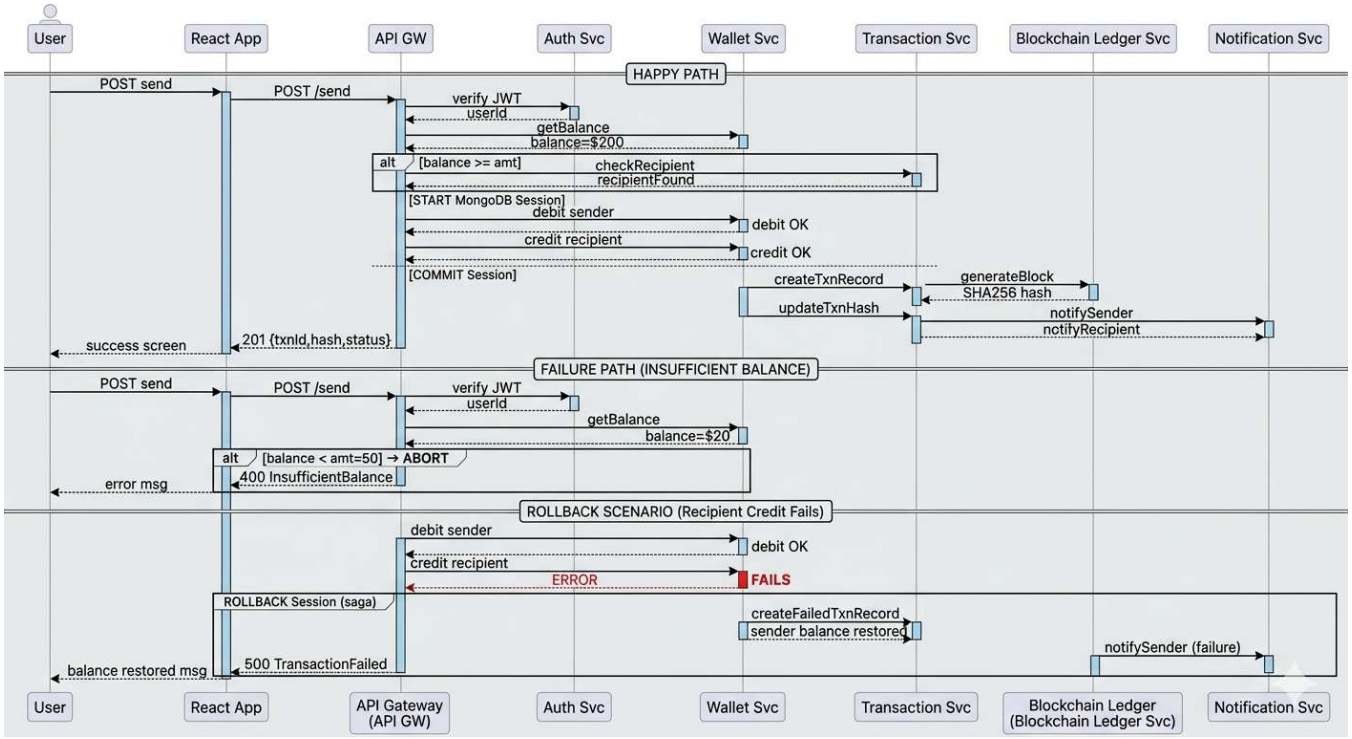
03 Class Diagram

Enterprise Entity Model · Inheritance · Composition · Multiplicity



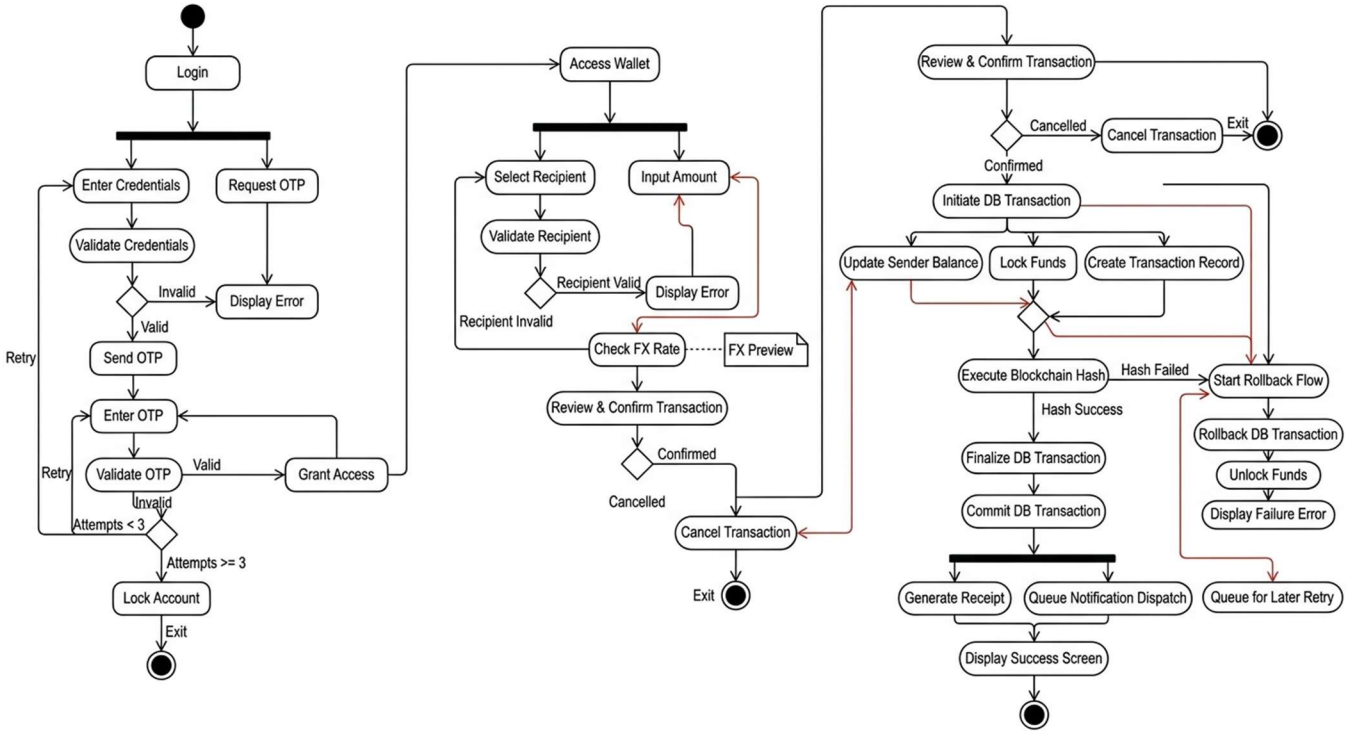
04 Sequence Diagram — Money Transfer

ACID Transaction Safety · Atomic Wallet Operations · Rollback Handling



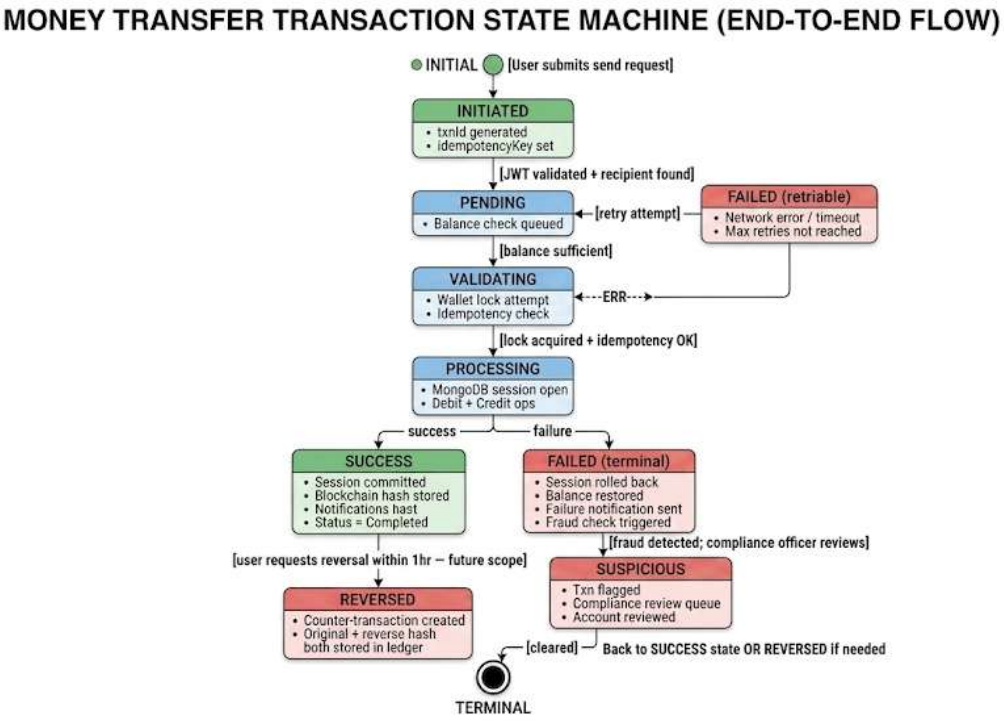
05 Activity Diagram — End-to-End Remittance Flow

Decision Nodes · Failure Paths · Retry Logic



06 State Machine Diagram — Transaction Lifecycle

7 States · Transition Triggers · Timeout Handling

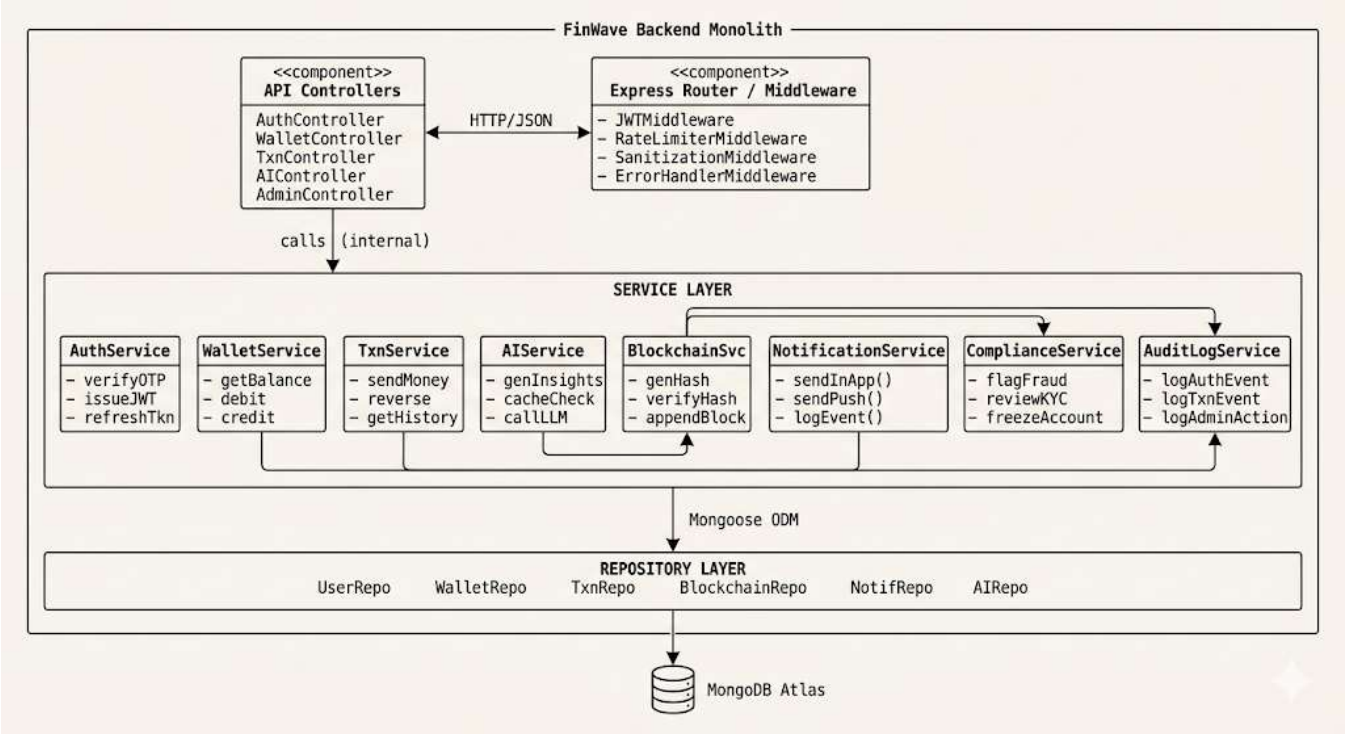


State Transition Table

Transition	Trigger / Condition
INITIATED → PENDING	JWT validated, recipient found, transaction object created with UUID
PENDING → VALIDATING	Sufficient wallet balance confirmed; idempotency key unique
VALIDATING → PROCESSING	Optimistic wallet lock acquired; MongoDB session opened
PROCESSING → SUCCESS	Both debit + credit committed; blockchain hash linked
PROCESSING → FAILED	Any DB error during session; saga rollback executed
SUCCESS → REVERSED	Reversal request within allowed window (future Phase 2)
FAILED → SUSPICIOUS	Fraud detection anomaly score exceeds threshold
PENDING/VALIDATING → FAILED	Timeout: no response within 30 seconds; lock expires

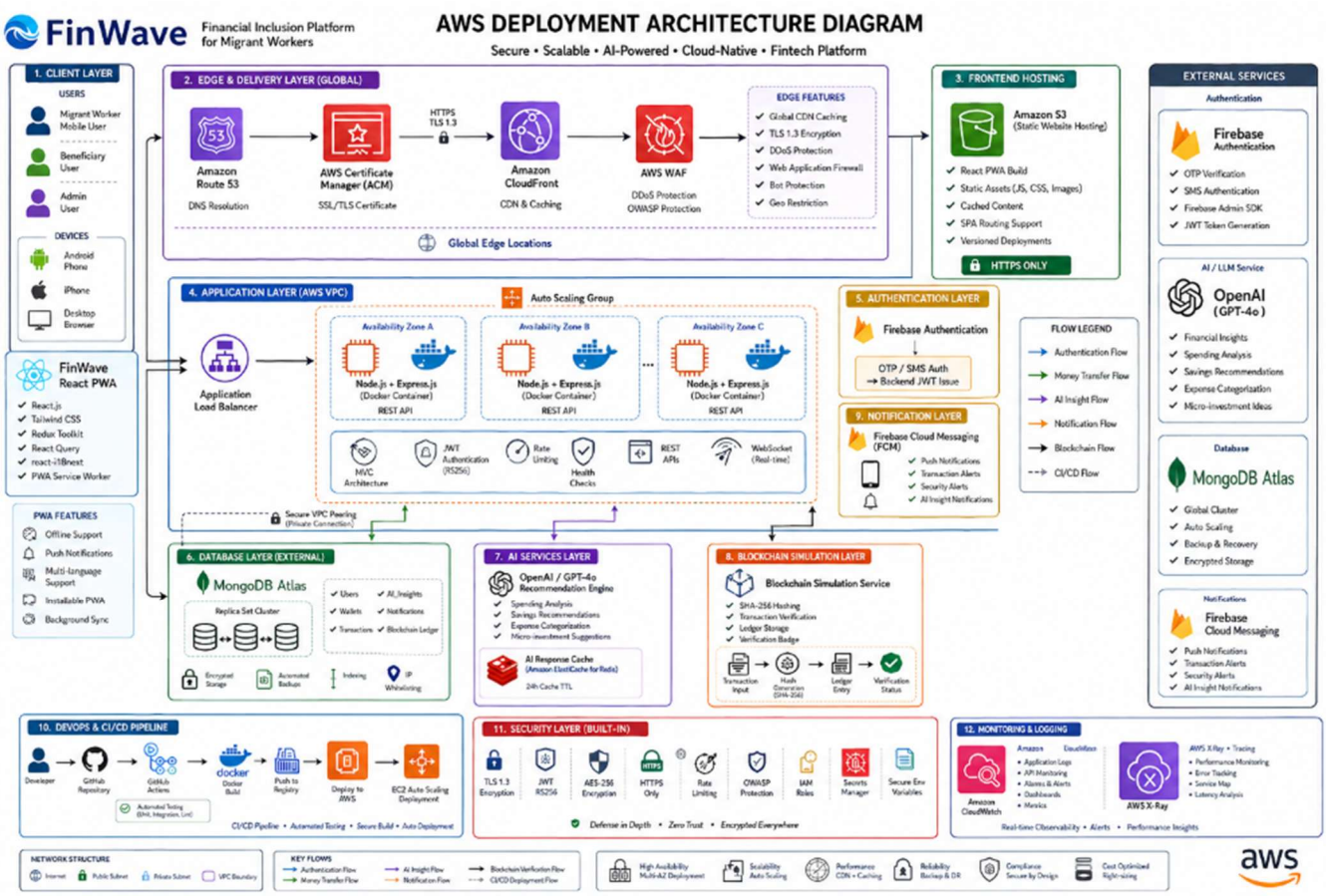
07 Component Diagram

Loose Coupling · Modular Monolith · Inteface Contracts



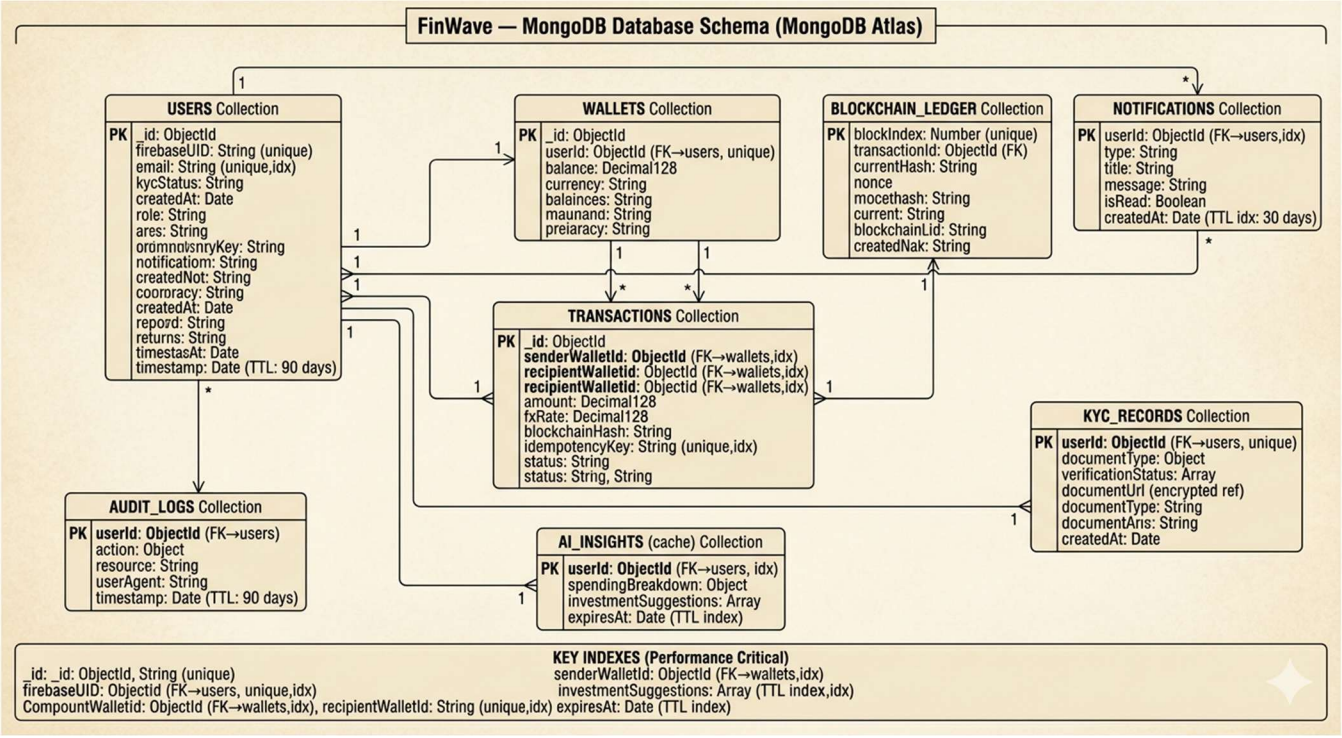
08 Deployment Diagram

Cloud-Native · Free-Tier MVP · Vercel + Render + MongoDB Atlas



09 Database ER Diagram

MongoDB Collections · Relationships · Indexes · Audit Trail

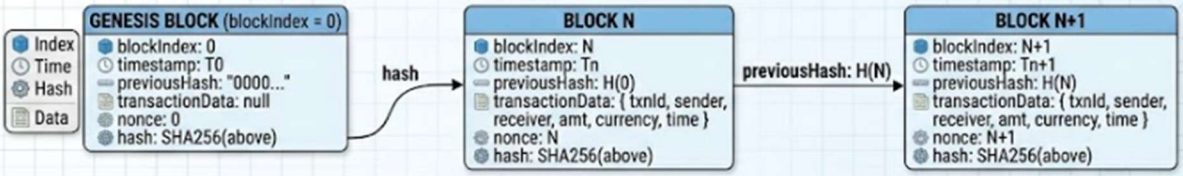


10 Data Flow Diagram — Levels 0, 1 & 2

Context DFD · Subsystem DFD · Process Detail DFD

FinWave Blockchain Simulation Module Architecture

Blockchain Simulation Flow



HASH COMPUTATION (Node.js crypto)

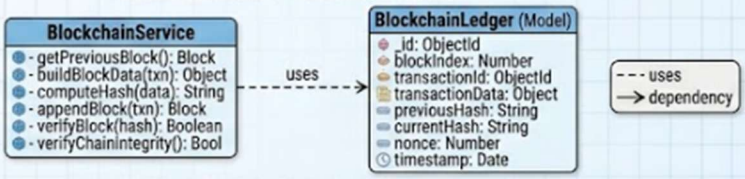
blockData = { blockIndex, timestamp (← ISO 8601 UTC), transactionData (← JSON.stringify with sorted keys), previousHash, nonce }

hash = crypto.createHash('sha256').update(JSON.stringify(blockData, Object.keys(blockData).sort())).digest('hex')

VERIFICATION FLOW

1. **Retrieve block from Blockchain collection by hash** [cite: x].
2. **Recompute hash from stored block data** [cite: x].
4. **Verify previousHash matches hash of (blockIndex - 1)** [cite: x].
5. **Return { verified: true/false, blockIndex, timestamp }** [cite: x].

CLASS DIAGRAM — Blockchain Module

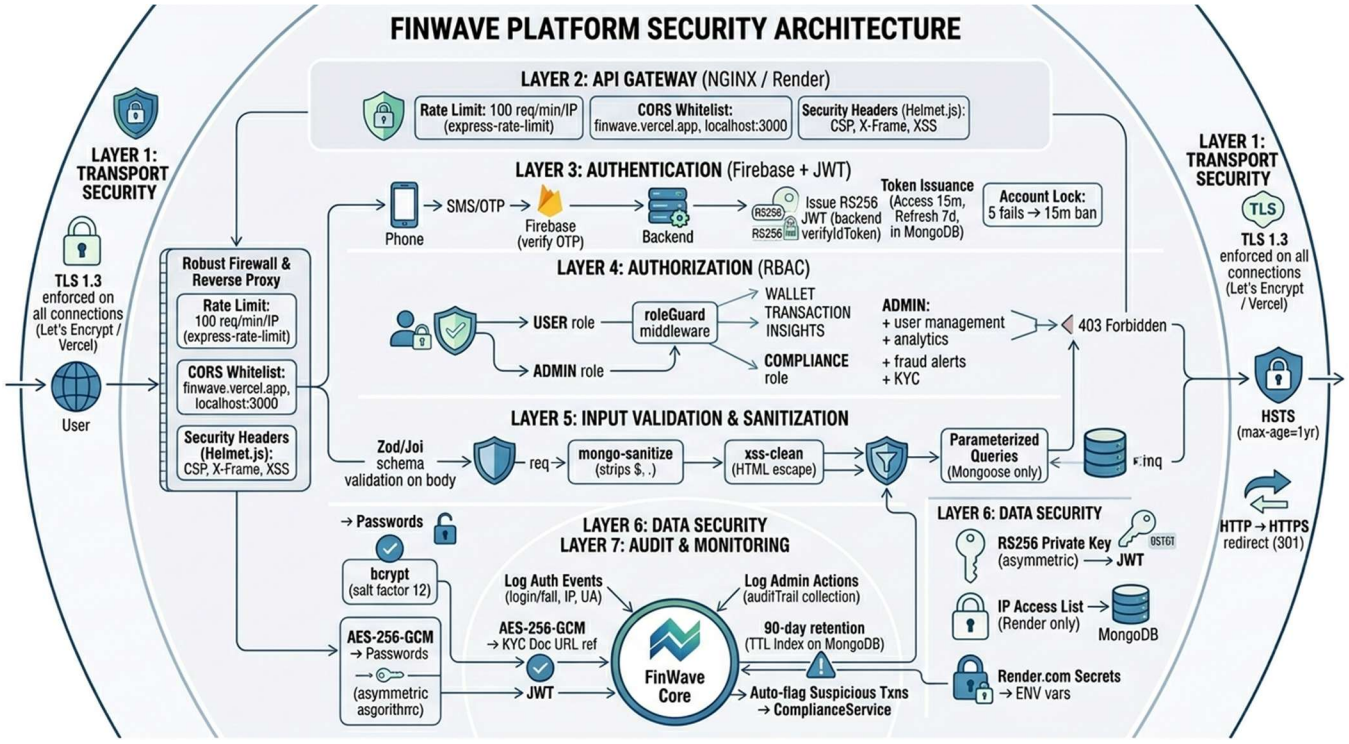


WHY NOT REAL BLOCKCHAIN? [cite: x]

- A 2-day hackathon MVP: SHA-256 chain in MongoDB gives users the **TRUST SIGNAL** (verifiable hash, tamper-evident chain) without complexity.
- Phase 3 can migrate to Polygon PoS.

11 Security Architecture Diagram

Defence-in-Depth · JWT RS256 · OWASP Top 10 Mitigation

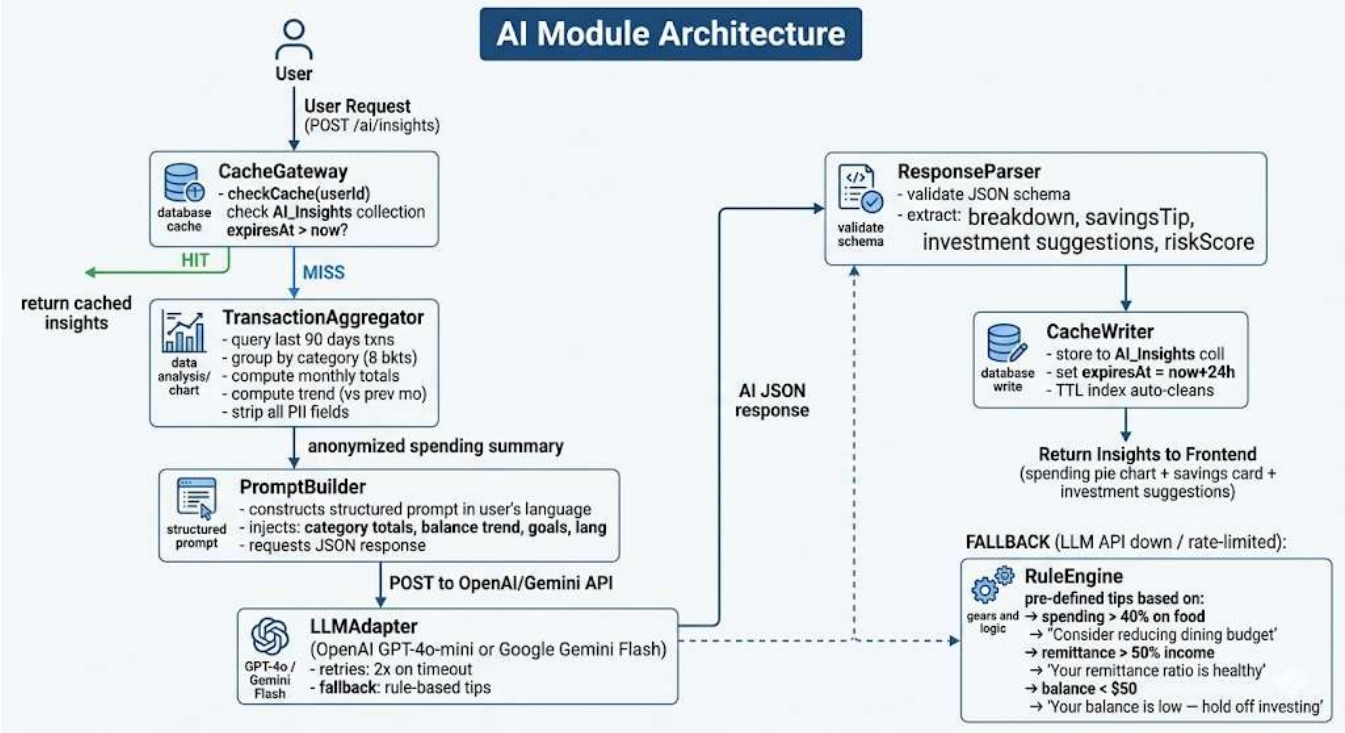


OWASP Top 10 — Mitigations

OWASP Risk	FinWave Mitigation
A01 – Broken Access Control	RBAC roleGuard on every route; 403 on unauthorized access; admin routes prefixed /admin
A02 – Cryptographic Failures	TLS 1.3 everywhere; bcrypt(12) passwords; AES-256 KYC docs; RS256 JWT
A03 – Injection	mongo-sanitize; Mongoose parameterized queries; Zod schema validation
A05 – Security Misconfiguration	Helmet.js headers; CORS whitelist; env vars in Render secrets
A07 – Auth Failures	Account lockout (5 attempts); OTP expiry (5min); JWT short expiry (15min)
A09 – Logging Failures	AuditLog collection; all auth events recorded with IP; 90-day TTL retention

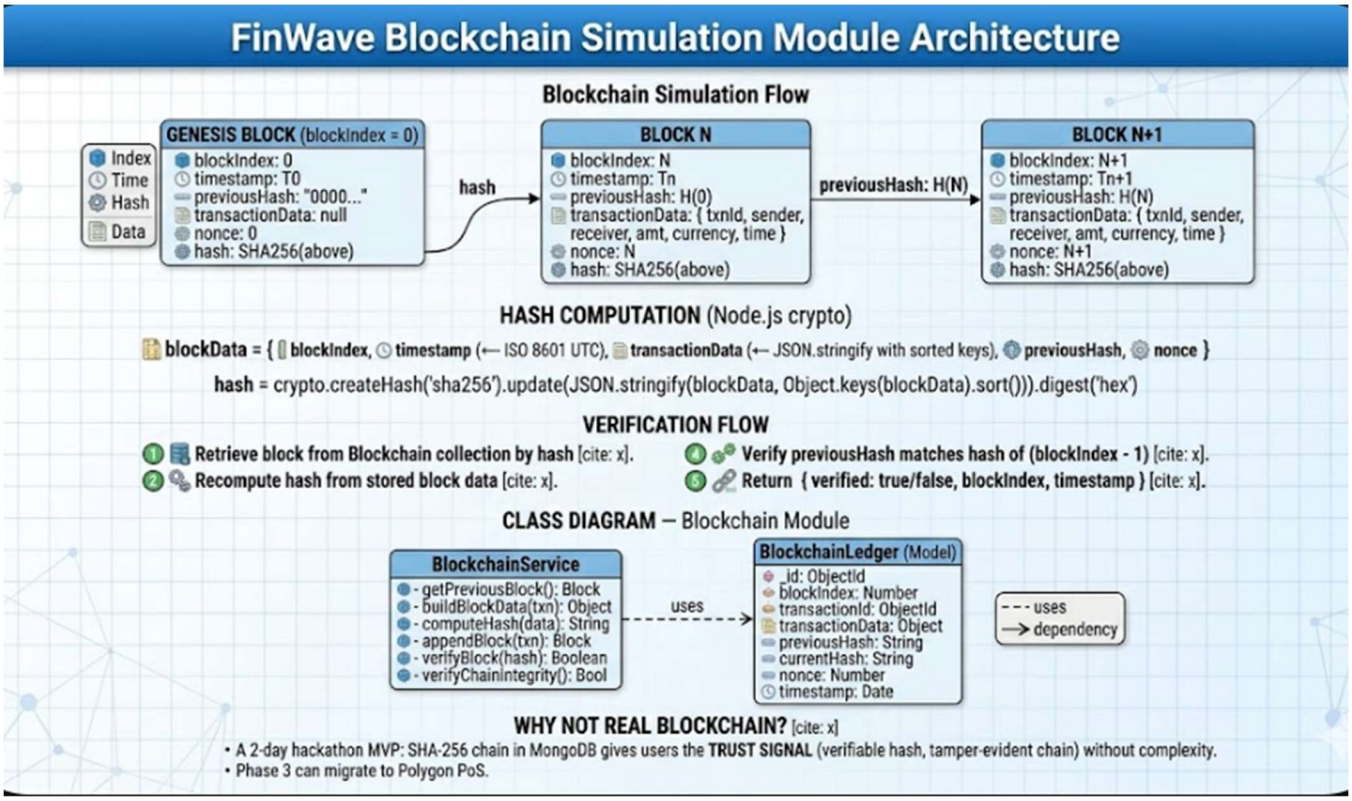
12 AI Module UML Design

Lightweight LLM Integration · 24h Cache · Rule-Based Hybrid



14 Blockchain-Inspired Ledger UML

SHA-256 Hashing · Immutable Chain · Verification Service



Design Justification Summary

Design Choice	Justification
SHA-256 via Node.js crypto	Built-in, zero dependencies, same algorithm used by Bitcoin — cryptographically sound for simulation.
MongoDB for ledger storage	Indexed on blockIndex for O(log n) chain traversal. TTL-free (blocks are permanent — no TTL index).
Sorted JSON serialization	Prevents hash inconsistency from JS object key ordering — deterministic hash for same data.
Chain linkage (previousHash)	If any block is tampered, all subsequent hashes become invalid — demonstrates immutability concept.
Nonce field	Simulates proof-of-work concept for educational value — incremented until hash meets simple criteria.

Design Suite Summary — 14 UML Diagrams Delivered

UML Diagram	Type & Key Focus
01 — High-Level Architecture	System Architecture · Modular monolith · Cloud tiers
02 — Use Case Diagram	Use Case · 6 actors · include/extend · fintech use cases
03 — Class Diagram	Class · Enterprise entities · Inheritance · Composition
04 — Sequence: Money Transfer	Sequence · ACID safety · Rollback handling
05 — Activity: Remittance Flow	Activity · Decision nodes · Retry logic · End-to-end
06 — State Machine: Transaction	State · 7 states · Timeout · Fraud path
07 — Component Diagram	Component · Layered monolith · Interface contracts
08 — Deployment Diagram	Deployment · Vercel + Render + Atlas · CI/CD
09 — ER Diagram (MongoDB)	ER · Collections · Indexes · Audit trail
10 — Data Flow Diagram (3 levels)	DFD L0/L1/L2 · Context → subsystem → process
11 — Package Diagram	Package · Clean architecture · Naming conventions
12 — Security Architecture	Security · Defence-in-depth · OWASP Top 10
13 — AI Module UML	AI Design · Cache · LLM adapter · Fallback
14 — Blockchain Ledger UML	Ledger · SHA-256 chain · Verification service